Co-otimização da Capacidade de Cache e da Técnica de *Tiling* em Aplicações HPC: Proposta de Pesquisa e Metodologia Experimental

Gustavo Parreira Epólita, João Comini César de Andrade, Rafael Rehfeld Martins de Oliveira

[1]Departamento de Ciência da Computação
Universidade [Nome da Instituição] – Belo Horizonte, MG – Brasil

{gustavo.epolita,joao.andrade,rafael.oliveira}@aluno.instituicao.br

Abstract. *This paper presents the second stage of a research proposal that investigates the interaction between very large 3D-stacked caches and the tiling (blocking) technique in High-Performance Computing (HPC) applications. Building on the gap identified in Domke et al. (2023), who evaluate the LARC processor without any software-level optimization aimed at exploiting its large L2 cache, we propose the hypothesis that tiling increases data reuse and can reduce cache misses in a way that partially substitutes the need for extremely large caches. We critically compare the base article with four related works, position our proposal against the state of the art, and describe an experimental methodology based on the gem5 simulator, using memory-bound HPC kernels executed with and without tiling under baseline and large-cache configurations.*

Resumo. *Este artigo apresenta a segunda etapa de uma proposta de pesquisa que investiga a interação entre caches 3D empilhadas de grande capacidade e a técnica de tiling (bloqueamento) em aplicações de Computação de Alto Desempenho (HPC). A partir da lacuna identificada em Domke et al. (2023), que avaliam o processador LARC sem nenhuma otimização em nível de software voltada a explorar sua grande cache L2, propomos a hipótese de que o tiling aumenta o reuso de dados e pode reduzir faltas de cache de forma a substituir parcialmente a necessidade de caches extremamente grandes. Comparamos criticamente o artigo-base com quatro trabalhos correlatos, posicionamos nossa proposta em relação ao estado da arte e descrevemos uma metodologia experimental baseada no simulador gem5, utilizando kernels HPC sensíveis à memória, executados com e sem tiling, sob configurações de cache de linha de base e de grande capacidade.*

1. Introdução

O desempenho de aplicações de Computação de Alto Desempenho (HPC) é frequentemente limitado pela capacidade da hierarquia de memória em fornecer dados aos núcleos de processamento na velocidade exigida pelos cálculos – o problema clássico da *parede de memória* [Wulf and McKee 1995]. Uma das estratégias mais discutidas atualmente para mitigar esse problema é o empilhamento tridimensional (3D) de memórias SRAM sobre o processador, ampliando drasticamente a capacidade de cache disponível próxima aos núcleos.

O artigo-base desta pesquisa, de Domke et al. [Domke et al. 2023], avalia essa estratégia por meio do processador hipotético LARC (*LARge Cache* processor), inspirado no Fujitsu A64FX do supercomputador Fugaku, mas projetado para um cenário tecnológico futuro com litografia de 1,5 nm e até 6 GiB de cache L2 SRAM empilhada. Usando análise de código e simulação detalhada em gem5, os autores mostram que, para aplicações com alta sensibilidade à memória, a capacidade extra de cache pode gerar acelerações médias de 1,9× por grupo de núcleos (CMG), chegando a até 9,56× sob escalonamento ideal para o chip completo.

Apesar dessas contribuições, o estudo apresenta uma lacuna relevante: as 52 aplicações analisadas na campanha principal de simulação são executadas praticamente sem qualquer reestruturação algorítmica voltada a explorar a supercache do LARC. Essa escolha metodológica é adequada para isolar o efeito do hardware, mas deixa aberta uma pergunta que os próprios autores não respondem: **em que medida uma melhor organização do software poderia reduzir a necessidade de uma cache tão grande?**

1.1. Pergunta de pesquisa e hipótese

A pergunta de pesquisa que guia este trabalho é: *em que medida a reorganização de aplicações HPC por meio da técnica de tiling pode reduzir faltas de cache e preservar o desempenho em arquiteturas com cache de alta capacidade inspiradas no LARC?*

O *tiling* (ou bloqueamento) consiste em dividir os dados e os cálculos de uma aplicação em blocos menores, de forma que cada bloco seja processado e reutilizado integralmente antes de ser substituído por novos dados na cache. Essa técnica é amplamente utilizada para aumentar a localidade temporal e espacial de acesso à memória, sobretudo em núcleos de álgebra linear densa e em métodos de diferenças finitas ou *stencil*, muito comuns em cargas de trabalho HPC.

A hipótese central deste trabalho é a seguinte: *a aplicação de tiling aumenta o reuso de dados enquanto estes permanecem na cache, reduzindo a taxa de faltas na cache L2 e o número de acessos à memória principal; como consequência, uma aplicação reorganizada em blocos e executada sob uma cache de capacidade menor tende a apresentar desempenho próximo – ou, em alguns casos, superior – ao de uma aplicação sem tiling executada sob uma cache muito maior, desde que o padrão de acesso da aplicação apresente reutilização de dados suficiente para ser explorada pelo bloqueamento.* Para padrões de acesso predominantemente *streaming*, com baixo reuso (por exemplo, operações vetoriais simples), espera-se que o ganho do *tiling* seja pequeno e que a capacidade de cache continue sendo o fator dominante.

1.2. Motivação e relevância

A motivação prática desta proposta está no compromisso entre investimento em hardware e otimização de software. Caches SRAM 3D na escala proposta pelo LARC implicam custos elevados de área, energia estática (Domke et al. estimam cerca de 98,3 W apenas para os 6 GiB de cache L2 de um chip completo) e complexidade de fabricação e resfriamento. Se parte do ganho observado com caches muito grandes puder ser reproduzida por meio de uma reorganização algorítmica de baixo custo, isso indica um caminho de otimização que não depende da evolução tecnológica de empilhamento 3D, sendo aplicável inclusive em arquiteturas já existentes.

A relevância científica da proposta decorre de três pontos. Primeiro, o artigo-base e a literatura correlata (discutida na Seção 2) convergem em apontar que o ganho de uma cache maior depende do padrão de reuso de dados da aplicação, mas nenhum desses trabalhos manipula diretamente esse padrão de reuso por meio de transformação de software com o objetivo específico de comparar contra o aumento de capacidade de cache. Segundo, caso a hipótese seja confirmada, o resultado ajuda a delimitar quais famílias de aplicações realmente justificam o investimento em cache 3D muito grande, contribuindo tanto para o projeto de arquiteturas quanto para o desenvolvimento de software HPC consciente de cache (*cache-aware*). Terceiro, mesmo que a hipótese seja apenas parcialmente confirmada, delimitar os padrões de acesso em que o *tiling não* compensa a falta de uma cache grande já constitui uma contribuição, pois orienta decisões de co-projeto entre aplicação e arquitetura.

1.3. Objetivos e contribuição

O objetivo geral deste trabalho é quantificar, por meio de simulação arquitetural, o efeito da técnica de *tiling* sobre a taxa de faltas de cache e o desempenho de *kernels* HPC sensíveis à memória, comparando esse efeito ao impacto isolado do aumento da capacidade da cache L2, na linha do que é investigado por Domke et al. [Domke et al. 2023] para o processador LARC. Como objetivos específicos, busca-se: (i) selecionar *kernels* representativos com e sem versões em blocos disponíveis na literatura; (ii) configurar cenários de simulação com capacidades de cache L2 de linha de base e de grande capacidade; (iii) definir métricas e um plano de análise que permitam comparar os quatro cenários resultantes (com/sem *tiling* × cache pequena/grande); e (iv) validar preliminarmente a viabilidade do ambiente de simulação escolhido. A contribuição esperada é uma evidência experimental sobre até que ponto uma otimização de software de baixo custo pode substituir parte do investimento em hardware de cache de grande capacidade em aplicações HPC.

2. Trabalhos Correlatos

Nesta seção comparamos criticamente o artigo-base com quatro trabalhos correlatos, publicados entre 2023 e 2026, que discutem sob perspectivas distintas a relação entre capacidade de cache, largura de banda de memória e desempenho em HPC. A Tabela 1 resume o posicionamento metodológico de cada trabalho.

Domke et al. [Domke et al. 2023] constituem o artigo-base e já foram discutidos na Seção 1. Sua principal contribuição metodológica é combinar um limite superior teórico (obtido por análise estática de código) com uma simulação ciclo a ciclo em gem5, permitindo separar o efeito do número de núcleos do efeito da capacidade de cache. A limitação central para esta proposta é que as 52 aplicações avaliadas são executadas sem qualquer transformação de software voltada a explorar a nova capacidade de cache – é exatamente essa lacuna que a presente pesquisa busca preencher.

Reguly [Reguly 2023] avalia aplicações limitadas por largura de banda em processadores Intel Xeon CPU Max Series reais, mostrando que o ganho de desempenho depende da compatibilidade entre o padrão de acesso da aplicação e a arquitetura de memória disponível. Em relação ao artigo-base, a vantagem de Reguly é usar hardware real, com resultados diretamente reprodutíveis; a desvantagem é não explorar caches

Table 1. Comparação metodológica entre o artigo-base e os trabalhos correlatos

Trabalho	Método	Foco	Limitação principal
Domke et al. [Domke et al. 2023]	Análise de código + simulação em gem5	Cache L2 3D muito grande (LARC)	Não avalia software otimizado nem o chip completo
Reguly [Reguly 2023]	Medição em hardware real	Largura de banda (Xeon Max)	Não avalia caches 3D nem arquiteturas futuras
Lawenda et al. [Lawenda et al. 2026]	Medição em hardware real	Cache L3 / 3D V-Cache em CFD	Ganho não uniforme entre tamanhos de problema
Rastegari et al. [Rastegari et al. 2026]	Medição em nuvem	Co-*design* aplicação–plataforma	Não isola o efeito específico da cache
Ghiasi et al. [Ghiasi et al. 2026]	Simulação de nova arquitetura	Integração lógica–memória	Reduz a necessidade de cache, não a explora via software

SRAM 3D de grande capacidade nem arquiteturas hipotéticas. Para esta proposta, o trabalho reforça que aplicações *memory-bound* respondem de formas distintas ao aumento de recursos de memória, o que motiva a escolha de múltiplos *kernels* com diferentes padrões de acesso em nossa metodologia.

Lawenda et al. [Lawenda et al. 2026] avaliam processadores AMD EPYC com diferentes capacidades de cache L3, incluindo modelos com 3D V-Cache, em aplicações de dinâmica dos fluidos computacional. A vantagem desse trabalho é validar em hardware comercial, já disponível, a ideia de que caches maiores beneficiam aplicações científicas – o mesmo fenômeno que Domke et al. projetam para um cenário futuro muito mais agressivo. A limitação relatada pelos próprios autores é que o ganho não é uniforme entre tamanhos de problema e configurações, o que é consistente com a hipótese desta proposta de que o benefício de mais cache depende do grau de reuso de dados, algo que o *tiling* manipula diretamente.

Rastegari et al. [Rastegari et al. 2026] investigam máquinas virtuais Azure HBv5 sob a perspectiva de co-*design* entre aplicação e plataforma, considerando cache, memória principal e rede em conjunto. A contribuição desse trabalho é ampliar a discussão para além da cache isolada, mas sua limitação, para os fins desta proposta, é não isolar experimentalmente o efeito específico da capacidade de cache dos demais fatores (rede, comunicação), o que o artigo-base e nossa proposta fazem de forma mais controlada via simulação.

Ghiasi et al. [Ghiasi et al. 2026] propõem a arquitetura RevaMp3D, que integra monoliticamente lógica e memória, questionando se investir área em uma cache de último nível muito grande é sempre a melhor decisão quando a memória principal tem baixa latência e alta largura de banda. Esse trabalho relativiza diretamente a premissa do artigo-base: em vez de ampliar a cache, ele reduz a distância até a memória principal. A relação com nossa proposta é conceitualmente semelhante – ambos questionam se "mais cache" é sempre necessário – mas por caminhos diferentes: Ghiasi et al. atacam o problema pelo lado do *hardware* (menor latência de memória principal), enquanto esta proposta ataca pelo lado do *software* (melhor reuso de dados via *tiling*).

2.1. Posicionamento da proposta

Em conjunto, os trabalhos correlatos indicam que (i) o benefício de mais cache não é uniforme e depende do padrão de acesso da aplicação [Reguly 2023, Lawenda et al. 2026]; (ii) esse benefício também depende de fatores além da cache, como latência e largura de banda da memória principal e comunicação entre unidades [Rastegari et al. 2026, Ghiasi et al. 2026]; e (iii) nenhum desses trabalhos, incluindo o artigo-base [Domke et al. 2023], manipula deliberadamente o padrão de reuso de dados via reestruturação de software para compará-lo ao efeito do aumento de capacidade de cache. É exatamente nesse espaço que a presente proposta se posiciona: como um estudo controlado, via simulação, do compromisso entre otimização de software (*tiling*) e capacidade de hardware (cache L2), usando o mesmo tipo de instrumentação (gem5) empregada no artigo-base, o que facilita a comparabilidade dos resultados.

3. Metodologia Experimental

3.1. Simulador

Optou-se pelo simulador arquitetural **gem5** [The gem5 Project 2024], pelos mesmos motivos que o tornaram adequado ao artigo-base: é um simulador de ciclo detalhado, amplamente utilizado em pesquisa de arquitetura de computadores, com suporte a diferentes modelos de CPU, hierarquias de cache configuráveis e modos de execução em emulação de sistema (SE) ou sistema completo (FS). Para este trabalho, será utilizado o modo SE, que dispensa a simulação de um sistema operacional completo e permite iterar mais rapidamente sobre as configurações de cache, o que é adequado dado o escopo e o prazo do projeto. A escolha do gem5, em vez do Sniper, também favorece a comparabilidade direta com os resultados do artigo-base, que utiliza o mesmo simulador em sua campanha principal.

3.2. Cargas de trabalho (*workloads*)

Para viabilizar a comparação direta entre versões com e sem *tiling* do mesmo cálculo, serão utilizados *kernels* de HPC que possuem, na literatura, tanto uma implementação de referência (*naïve*) quanto uma implementação em blocos:

- **PolyBench/C** [Pouchet et al. 2012]: conjunto de *kernels* numéricos (por exemplo, multiplicação de matrizes – GEMM, e o método iterativo *stencil* `jacobi-2d`) que já inclui, para vários casos, variantes com bloqueamento gerado por otimização polihédrica, facilitando a obtenção de pares comparáveis (com/sem *tiling*).
- **NAS Parallel Benchmarks (NPB)** [Bailey et al. 1991]: em particular o *kernel* MG (multigrid), citado no próprio artigo-base como uma das aplicações que mais se beneficiam da cache adicional, com taxa de faltas caindo de mais de 30% para valores próximos de zero no LARC. Uma versão em blocos do MG será implementada manualmente a partir do código de referência, permitindo comparação direta com os resultados relatados por Domke et al.

Cada *kernel* será executado em dois tamanhos de problema (um que caiba confortavelmente na cache de linha de base e outro que exceda essa capacidade), de forma a caracterizar o comportamento da aplicação em diferentes regimes de localidade.

3.3. Parâmetros arquiteturais

Serão simuladas duas configurações de cache L2 por núcleo, inspiradas na comparação entre o A64FX (linha de base) e o LARC (cache expandida) do artigo-base, porém em escala compatível com o tempo de simulação disponível no gem5 em modo ciclo a ciclo:

- **Cenário Base:** L1 de 32 KiB (dados) + 32 KiB (instruções) e L2 compartilhada de 8 MiB, refletindo a configuração por CMG do A64FX relatada no artigo-base.
- **Cenário Cache Grande:** mesma L1, com L2 ampliada para 128 MiB. Esse valor foi escolhido por ser o maior praticável em simulação de ciclo detalhado no tempo disponível do projeto, mantendo a mesma *razão de crescimento qualitativa* (aumento de mais de uma ordem de grandeza na capacidade da L2) usada pelo LARC em relação ao A64FX, sem pretender reproduzir os 384–512 MiB por CMG do artigo-base, que exigiriam recursos computacionais fora do escopo deste trabalho.

Núcleo simulado do tipo *out-of-order*, modelo `DerivO3CPU` do gem5, com *clock* e parâmetros de emissão fixos entre as configurações, de forma que a única variável arquitetural alterada entre os dois cenários seja a capacidade da L2.

3.4. Variáveis independentes e dependentes

O desenho experimental segue um esquema fatorial 2×2:

- **Variáveis independentes:** (1) uso de *tiling* (com *vs.* sem) e (2) capacidade da cache L2 (Base *vs.* Grande).
- **Variáveis dependentes:** taxa de faltas de cache na L2 (*miss rate*); número de acessos à memória principal; *Instructions Per Cycle* (IPC); e tempo de execução simulado (em ciclos), do qual deriva o *speedup* relativo ao cenário Base sem *tiling*.

Essa combinação resulta em quatro cenários por *kernel* e tamanho de problema: (Base, sem *tiling*), (Base, com *tiling*), (Grande, sem *tiling*) e (Grande, com *tiling*). A hipótese será considerada corroborada, para um dado *kernel*, se o cenário (Base, com *tiling*) apresentar *miss rate* e tempo de execução próximos aos do cenário (Grande, sem *tiling*) – isto é, se o *tiling* sob cache pequena aproximar o desempenho obtido apenas por meio de mais cache.

3.5. Estratégia de análise dos resultados

Para cada *kernel* e tamanho de problema, será calculado o *speedup* de cada um dos quatro cenários em relação ao cenário (Base, sem *tiling*), seguindo a mesma lógica de comparação relativa usada no artigo-base. Quando houver mais de um *kernel* da mesma família, será reportada também a média geométrica dos *speedups*, como feito por Domke et al. [Domke et al. 2023] ao agregar resultados entre aplicações. A análise central consistirá em comparar, para cada *kernel*, a diferença relativa entre o *speedup* de (Base, com *tiling*) e o de (Grande, sem *tiling*): diferenças pequenas favorecem a hipótese; diferenças grandes e persistentes indicam que, para aquele padrão de acesso, a capacidade de cache permanece o fator dominante e o *tiling* não é suficiente para substituí-la.

3.6. Ambiente experimental e reprodutibilidade

Todos os experimentos serão executados com uma versão fixa e documentada do gem5 (branch estável mais recente no momento da Etapa 3), compilada a partir do código-fonte oficial disponível no repositório do projeto. Os *scripts* de configuração de simulação (topologia de cache, parâmetros de CPU, comandos de execução de cada *kernel*) e o código-fonte das versões com e sem *tiling* de cada *kernel* serão versionados em um repositório do grupo, junto com os arquivos de saída (estatísticas do gem5) de cada execução, de modo a garantir que qualquer execução possa ser reproduzida a partir do mesmo *commit*. Cada configuração será executada até a conclusão determinística do *kernel* (sem amostragem aleatória de entrada), o que torna os resultados naturalmente reprodutíveis entre execuções, dispensando repetições múltiplas para controle de variância – exceto quando o próprio gem5 introduzir não determinismo (por exemplo, em simulação multi-thread), caso em que serão realizadas três repetições e reportada a média.

3.7. Testes preliminares

Como validação inicial da viabilidade da metodologia, o grupo já iniciou a configuração do ambiente gem5 em modo SE, compilando o simulador para a arquitetura ARM (compatível com o A64FX do artigo-base) e executando um *kernel* de multiplicação de matrizes de tamanho reduzido como teste de fumaça (*smoke test*), a fim de confirmar que o ambiente produz estatísticas de cache (*miss rate*, acessos) coerentes com o esperado antes de escalar para os *kernels* e tamanhos de problema definitivos, que serão utilizados na coleta de resultados da Etapa 3.

4. Considerações Finais desta Etapa

Esta etapa consolidou a oportunidade de pesquisa identificada na Etapa 1 em uma proposta experimental concreta: investigar se a técnica de *tiling* pode reduzir, ao menos parcialmente, a necessidade de caches L2 extremamente grandes como a proposta pelo processador LARC [Domke et al. 2023]. A comparação crítica com quatro trabalhos correlatos mostrou que, embora a relação entre capacidade de cache e desempenho em HPC já seja bem documentada, nenhum desses trabalhos manipula diretamente o reuso de dados via reestruturação de software para compará-lo ao efeito do hardware – lacuna que esta proposta busca preencher. A metodologia baseada em gem5, com um desenho fatorial 2×2 (tiling × capacidade de cache) sobre *kernels* de PolyBench e NPB, fornece um plano experimental reprodutível cujos resultados serão apresentados na Etapa 3.

References

Bailey, D. H. et al. (1991). The nas parallel benchmarks. *International Journal of High Performance Computing Applications*, 5(3):63–73.

Domke, J., Vatai, E., Kraus, J., Drozd, A., Chen, P., Adachi, Y., and Yokota, R. (2023). At the locus of performance: Quantifying the effects of copious 3d-stacked cache on hpc workloads. *ACM Transactions on Architecture and Code Optimization*, 20(4):1–26.

Ghiasi, N. M. et al. (2026). Revamp3d: Architecting the processor core and cache hierarchy for systems with monolithically-integrated logic and memory. *ACM Transactions on Architecture and Code Optimization*, 23(3):1–26.

Lawenda, M. et al. (2026). Evaluating amd epyc cpu architectures on cfd applications. *Future Generation Computer Systems*, 177.

Pouchet, L.-N. et al. (2012). Polybench/c: The polyhedral benchmark suite. `https://sourceforge.net/projects/polybench/`. Acesso em: set. 2026.

Rastegari, A. et al. (2026). Cloud-hardware co-design for memory bandwidth-bound hpc workloads: Performance and characteristics of azure hbv5 virtual machines. In *International Conference on High Performance Computing in Asia-Pacific Region*, pages 247–257.

Reguly, I. Z. (2023). Comparative evaluation of bandwidth-bound applications on intel xeon cpu max series. In *SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, pages 1236–1244.

The gem5 Project (2024). gem5: The gem5 simulator. `https://www.gem5.org`. Acesso em: set. 2026.

Wulf, W. A. and McKee, S. A. (1995). Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1):20–24.